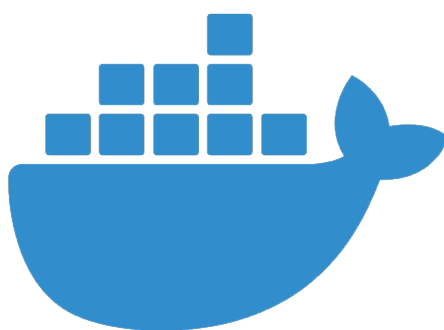
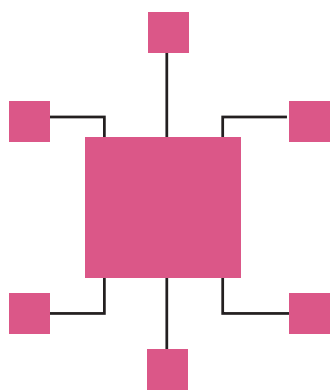


Things to Know About Microservices and Containerisation

Here's a quick look at how microservices and containerisation work, with a few best practices thrown in.



black box (no internal details are exposed outside the service boundary) for other systems using them.

Bounded context: This is one of the most important characteristics of microservices. Other services do not need to know anything about the architecture of a microservice.

Innovation friendly: New versions of small independent services can be deployed rapidly, making it easier to experiment and innovate.

Quality: Composability and responsibility, more maintainable code, better scaling and optimisation are all characteristics of microservices.

End-to-end ownership: Microservices bring about a cultural shift for the project team. The development team maintains each project built using microservices architecture as opposed to the regular practice of handing it over to the support and maintenance team.

Independent governance: A single team has the complete ownership of the microservice lifecycle. It owns all the aspects of the microservices including governance. Governance is decentralised and autonomous.

Manageability: Smaller microservices code bases are easier for developers to understand, making changes and deployments easier.

Containers

Containers are packages of software that include everything that they need

Microservices can be developed in any programming language. Each microservice focuses on completing one task and there are no criteria as to how small it should be. These services help to scale up parts of applications rather than scaling up the entire application. Major characteristics of microservices are:

Service independence: As microservices are autonomous, each microservice can change independent of other services.

Self-contained: Microservices are easily replaceable and upgradeable and promote single ownership of the service.

Decentralised: Microservices decentralise patterns, languages, standards, as well as component and data responsibility and governance because of their architectural style.

Resiliency: If one microservice goes down, it does not bring the whole application down.

Polyglot architecture: Microservices architecture supports multiple technologies.

Single responsibility: A service is aligned to a single business activity and is responsible for that. It delivers the complete business logic necessary to fulfill that business activity.

Scalability: Microservices can be deployed and scaled independent of each other.

Decoupled: Microservices implement a single business function. This helps in maintaining minimum dependency on other services.

Implementation agnostic: Microservices support multiple development platforms and technologies and can be deployed in any of the containers.

Blackbox: Microservices are a